HLS implementation of an autoencoder scheme

ELEC-Y541 - Reconfigurable architectures

Emmeran Colot

2026

1 Introduction

The objective of this report is to showcase the work done in an effort to implement an autoencoder architecture on an FPGA using high-level synthesis (HLS) tools. The model uses the MNIST dataset, consisting of 28x28 black and white pixels of handwritten numbers, and attempts to compress those images while minimizing the difference between the original image and the reconstructed one.

A trained model, written with PyTorch, was already given and only inference had to be run on the final implementation. The model first had to be translated in C++, which is explained in Sec. 2. The resulting implementation then has been tuned with different synthesis directives, optimizing either the performances in Sec. 3 or the area in Sec. 4.

2 C++ Implementation

2.1 Auto-encoder Model Structure

The model consists of 4 encryption layers and 4 decryption layers. Each layer consists of a linear part where the input is multiplied by a *weight* matrix, and then added to a *bias* vector. This linear operation is then followed by an activation function, which is a rectified linear unit (ReLU) for all layers except for the output layer.

Each layer can therefore be implemented by slightly calling `Layer` (given below) with the right sizes and weights/biasses. It can be mathematically summarized as

$$y_i = \begin{cases} 0 & \text{if} \quad (\mathbf{W}_{(\mathbf{i},:)}\mathbf{u} + b_i) < 0 \\ \mathbf{W}_{(\mathbf{i},:)}\mathbf{u} + b_i & \text{otherwise,} \end{cases} \tag{1}$$

where $\mathbf{u}$ and $\mathbf{y}$ are respectively the input and output vector, $\mathbf{W}$ the weight matrix and $\mathbf{b}$ the bias vector.

2.2 Layer Specialization

A first issue that arose from the basic approach of a single function called for every layer is the dynamic number of iterations in the **L0** and **L1** loops, which prevented loop unrolling. For this

```
void Layer(float layerInput[MAX_SIZE],
        float layerOutput[MAX_SIZE],
        float weights[MAX_SIZE][MAX_SIZE],
        float bias[MAX_SIZE],
        int input_size,
        int output_size)
    {
    L0:for(int row = 0; row < output_size; row++)
        {
        float tmp = bias[row];  // Start with bias
        L1:for(int col = 0; col < input_size; col++)
        {
            tmp = tmp + weights[row][col] * layerInput[col];
        }
        // ReLu
        layerOutput[row] = max(0.0f, tmp);
        }
    }
```

reason, each type of layer[1] has a specific function with fixed sizes.

2.3 Parameters Sharing

A first version of the model passed weights and biasses to the function to accelerate as parameters. It was found in the optimization process that such method led to complicated interface handling. This has therefore been changed to pass the model parameters through a header file.

3 Performance Optimization

3.1 Inner Pipelining

The first step taken towards a performance-focussed implementation was to try pipelining the inner loop **L1**. However, because the model runs with a FP32 format, additions and multiplications are slow and this led to an initiation interval (II) of 3 and a clock period higher than the default one ($\approx$ 13.3s). This is not an issue and the desired clock frequency simply has to be set to a larger value in order for the model to respect the constraints.

3.2 Outer Pipelining

By pipelining the outer loop **L0**, each element of the output vector is computed in parallel. This is a significant improvement of the interval time.

4 Area Optimization

[1] a *type of layer* designates a layer with a given input and output layer size, but 2 layers can still have the same function if their sizes match.